Static vs Non-Static Methods

static methods vs non static methods

In Java, methods can either be **static** or **non-static** based on their behavior and usage. Here's a concise comparison:

Static Methods

- **Definition**: Declared using the `static` keyword.
- **Belonging**: Belong to the **class** rather than an instance of the class.
- **Access**: Can be called without creating an object of the class (e.g., `ClassName.methodName()`).
- **Access Rules**:
  - Can **only access static members** (methods or variables) directly.
  - Cannot access instance (non-static) members unless an object is explicitly created.
- **Use Case**: Used for utility or helper methods (e.g., `Math.sqrt()`), or when the functionality does not depend on object-specific data.

Example:

```java
class Example {
    static void staticMethod() {
        System.out.println("This is a static method.");
    }
}

public class Main {
    public static void main(String[] args) {
        Example.staticMethod(); // Called without creating an object
    }
}
```

Non-Static Methods

- **Definition**: Do not have the `static` keyword.
- **Belonging**: Belong to an **object** of the class.
- **Access**: Require an instance (object) of the class to be called (e.g., `object.methodName()`).
- **Access Rules**:
  - Can access **both static and non-static members** directly.
  - Typically work with instance variables specific to the object.
- **Use Case**: Used when the method needs to operate on the specific instance's data or behavior.

Example:

```java
class Example {
    void nonStaticMethod() {
        System.out.println("This is a non-static method.");
    }
}

public class Main {
    public static void main(String[] args) {
        Example obj = new Example(); // Create an object
        obj.nonStaticMethod();       // Call using the object
    }
}
```

Key Differences

Aspect	Static Method	Non-Static Method
Belongs To	Class	Object
Call Requires	Class name (or directly within the class)	Object of the class
Access to Members	Only static members directly	Both static and non-static members
Use Cases	Utility, independent logic	Instance-specific behavior or data

static methods Can only access static members (methods or variables) directly.

non static method Can access both static and non-static members directly.
explain with program

Here's an example program to demonstrate the access rules of static and non-static methods in Java:

Program Explanation

1. A **static method** can only directly access **static members** (variables or methods).
   - To access **non-static members**, an object of the class must be created.
2. A **non-static method** can directly access both **static** and **non-static members**.

Code Example

```java
class Example {
    // Static member variables
    static int staticVariable = 100;

    // Non-static member variables
    int nonStaticVariable = 200;
```

```
    // Static method
    static void staticMethod() {
        // Access static variable directly
        System.out.println("Static Variable: " + staticVariable);

        // Access non-static variable (NOT directly, needs an object)
        Example obj = new Example();
        System.out.println("Non-Static Variable (via object): " + obj.nonStaticVariable);
    }

    // Non-static method
    void nonStaticMethod() {
        // Access static variable directly
        System.out.println("Static Variable: " + staticVariable);

        // Access non-static variable directly
        System.out.println("Non-Static Variable: " + nonStaticVariable);
    }
}

public class Main {
    public static void main(String[] args) {
        // Call static method directly using class name
        Example.staticMethod();

        // Create an object of the class
        Example obj = new Example();

        // Call non-static method using the object
        obj.nonStaticMethod();
    }
}
```

Output

```
vbnet


Static Variable: 100
Non-Static Variable (via object): 200
Static Variable: 100
Non-Static Variable: 200
```

Explanation of Access

1. **In the Static Method:**
   - `staticVariable` is accessed directly because it's a static member.
   - `nonStaticVariable` cannot be accessed directly; an object `obj` of the class must be created to access it.
2. **In the Non-Static Method:**
   - `staticVariable` is accessed directly because static members can be used anywhere.
   - `nonStaticVariable` is accessed directly because it's within the context of the object.

This demonstrates how **static methods** and **non-static methods** differ in accessing class members.